VIO-X1

INTELLIGENT VIDEO INTERFACE

USER MANUAL

ADDENDUM C1

9/20/82
#295 AT

ADDENDUM C1 11/10/81

For firmware versions C1 and later.

PLEASE UPDATE YOUR REV A1 MANUAL

FULCRUM™
COMPUTER PRODUCTS
Distributed by:
WW COMPONENT SUPPLY INC. 1771 JUNCTION AVENUE • SAN JOSE, CA 95112 • (408) 295-7171

Software Interface

There are three different ways for the host to make the terminal do something:

1) Send a normal character.
2) Send a control character.
2a) Send an escape sequence.

1) Normal Characters

There are 96 normal characters, i.e., the entire ASCII character set. Transmitting any of these characters to the terminal results in the character appearing on the screen at the cursor position. The cursor then moves one position to the right. If the cursor is at the right side of the screen when this occurs, it wraps to the left side, moving down one line. If the cursor is at the bottom right corner, it will move to the bottom left corner and the screen will scroll up, leaving a blank line at the bottom.

2) Control characters

There are 32 control characters in the ASCII character set. The terminal responds to some (but not all) of them. The characters which the terminal notices are:

Char	Hex	Meaning	Action Taken
@	00H	NULL	Ignored by terminal ***
D	04H	ROLL DOWN	Scrolls screen down one line
G	07H	BELL	Beeps Beeper
H	08H	BACKSPACE	Bumps cursor to the left
J	0AH	LINE FEED	Bumps cursor down one line
K	0BH	CURSOR UP	Bumps cursor up one line
L	0CH	CURSOR RIGHT	Bumps cursor to the right
M	0DH	CAR RETURN	Returns cursor to left side
U	15H	ROLL UP	Scrolls screen up one line
Z	1AH	CLEAR SCRN	Clears the screen to blanks
[	1BH	ESCAPE	Initiates an escape sequence
^	1EH	HOME	Returns cursor to upper left
_	1FH	NEW LINE	Combined CR/LF

While the terminal does not respond to any other control characters, you can generate any of them by holding down CTRL and hitting the appropriate key. The terminal will then transmit that control character to the host.

Differences between VIO-X rev A1 and C0:

1) Video attributes are now handled on a character basis, rather than the fields of rev A1.

2) Insert-line and delete-line have been added.

3) A soft reset capability from the keyboard has been added.

4) The total number of lines contained in the terminal's memory has dropped from 49 to 41.

5) Send-line and send-page have been deleted.

VIO-X
INTERRUPT STRAPPING LIST

PIN#	SIGNAL	DESCRIPTION
01	V7	VECTORED INTERRUPT 7
02	V6	VECTORED INTERRUPT 6
03	V5	VECTORED INTERRUPT 5
04	V4	VECTORED INTERRUPT 4
05	V3	VECTORED INTERRUPT 3
06	V2	VECTORED INTERRUPT 2
07	V1	VECTORED INTERRUPT 1
08	V0	VECTORED INTERRUPT 0
09	INT	INTERRUPT
11	DAV	
13	TBE	
15	DRDY	

Write to 25th line (ESC 8)

The terminal responds to ESC 8 by clearing the 25th line of the display. All subsequent incoming characters are the written to the 25th line instead of the cursor position (the cursor does not move during this time). Only normal characters are allowed. Any control character (including ESC) terminates the sequence. Up to 60 characters can be written to the 25th line.

One video attribute is allowed on the 25th line. It must have been set (via ESC 9) prior to starting the ESC 8 sequence. The entire message written to the 25th line will therefore have this attribute.

Writing to the 25th line is normally terminated by a carriage return character, although any of the conditions listed above will do it. Once the termination occurs, operation returns to normal.

To clear the 25th line, send ESC 8 <cr>.

Line 25 Att (ESC 9 <att>)

ESC 9 <att> sets the 25th line video attribute. This is the attribute in which characters written to the 25th line will appear.

On power-up or reset, the 25th line attribute is initialized to "normal."

Insert Line (ESC E)

When the terminal receives ESC E, it responds by pushing all lines, from the cursor to the bottom of the screen, down one line. This leav the cursor sitting on a blank line. The bottom line falls off the display and is lost.

Please note that if a scroll-down was done prior to the insert-line, there will be one or more lines below the bottom of the screen. These lines are not affected by insert line and can still be scrolled up as usual.

Delete line (ESC R)

ESC R causes the terminal to delete the line on which the cursor is sitting. This naturally sucks the lines below up one line, leaving a blank line at the bottom of the screen. Like insert-line, any lines below the bottom of the screen are unaffected by delete-line.

Set Transparent Mode (ESC U)

When the terminal receives ESC U, it reacts by setting Transparent mode, which is indicated by the message XPARENT in bright inverse characters on the 25th line. In this mode, all incoming characters, control or not, are displayed on the screen. No control characters cause any special action (one exception -- see below); e.g., carriage return does not perform the normal carriage return function, but displays a special character on the screen instead.

The sequence ESC ESC is recognized by the terminal and terminates transparent mode. The message in the status line is removed and the terminal returns to normal.

Set Video Attribute (ESC Z <att>)

ESC Z causes the terminal to treat the next incoming character as a video attribute. All subsequent characters will be written to the screen in this attribute, regardless of the attributes of the characters in the vicinity of the cursor. Say, for example, the active attribute is set to dim via ESC Z ! or ESC). If the cursor is positioned in the middle of a line of normal characters and a character (say "A") is transmitted to the terminal, a dim "A" will appear, surrounded by bright characters.

To the hardware, each different attribute which exists on a line comprises a different "field." Before the dim "A" was written, the line had only one field, whose attribute was normal. After the dim "A", the line now has three fields: the first is normal, up to the "A"; the second is dim, and contains only the "A"; and the third is normal again, from the charcter following the "A" down to the end of the line.

This concept is important because the number of fields that can exist in a line at one time is limited to 16 by the video chip. It is entirely possible to generate more than 16 fields (up to 80 could in fact be generated) because, in the interest of reasonable performance, the terminal firmware does not check the number of fields in the line. (Even with such a check, you could not get the result you expected by generating more than 16 fields; thus, correct system software would never try to do it). Feel free to experiment with more than 16 fields and see the interesting but harmless malfunctions that will occur (clearing the screen will return things to normal).

Note that the occurrence of two contiguous fields of the same attribute is possible (hardware-wise), but the firmware prevents this from ever happening.

The video attribute as sent by the host is a character in the range " " to "/" (20H to 2FH). The bit assignment is as follows:

```
        7   6   5   4   3   2   1   0

        0   0   1   0   U   I   B   D
        |   |   |   |   |   |   |   |
        |   |   |   |   |   |   |   +-----Sets Dim attribute
        |   |   |   |   |   |   +---------Sets Blinking attribute
        |   |   |   |   |   +-------------Sets Inverse attribute
        |   |   |   |   +-----------------Sets Underline attribute
        |   |   |   +---------------------\
        |   |   +------------------------- \ usually = 0010, but
        |   +----------------------------- / it doesn't really matter
        +---------------------------------/
```

Char	Hex	Resulting Attribute
space	20H	Normal
!	21H	Dim
"	22H	Blinking
#	23H	Blinking dim
$	24H	Inverse
%	25H	Inverse dim
&	26H	Inverse blinking
'	27H	Inverse blinking dim
(	28H	Underline
)	29H	Underline dim
*	2AH	Underline blinking
+	2BH	Underline blinking dim
,	2CH	Underline inverse
-	2DH	Underline inverse dim
.	2EH	Underline inverse blinking
/	2FH	Underline inverse blinking dim

Thus, to set the active attribute to inverse, the host sends ESC Z $ ("$" is 24H, the code for an inverse video attribute).

Read Version (ESC DEL)

When the terminal receives ESC DEL (7FH), it returns a two character sequence which represents the version number of the firmware ROM currently installed in the terminal. The first character is a letter A-Z and indicates the microcode type. The second character is a digit 0-9 and indicates the revision level of the microcode.

Bright Att On [ESC ( ]

This escape sequence sets the current video attribute to "normal," i.e., bright, non-blinking, non-inverse, non-underline.

Dim Att On [ESC)]

This escape sequence sets the current video attribute to dim, non-blinking, non-inverse, non-underline.

Status On (ESC ,)

This escape sequence turns on the 25th line status field. In addition, it re-displays the clock if it is running.

Status Off (ESC .)

This turns the 25th line status field off. The user area of the 25th line is unaffected; however, the status display and the clock (if running) are both blanked (the clock keeps running though, and is still readable by the system).

Erase Line (ESC T)

ESC T clears the current line from the cursor to the end of the line, inclusive, to normal video blanks.

Erase Page (ESC Y)

ESC Y clears the screen from the cursor to the end of the screen, inclusive, to normal video blanks. If there are lines below the screen (due to a roll- down), they are unaffected by erase-page.

The VIO-X has no control over what type of keyboard is plugged into it. However, in order to function properly the keyboard must meet certain requirements.

Naturally, its connector must match the VIO-X pinout (refer to the hardware description and schematic). The IMSAI IKB-1 is compatible with this pinout.

If your keyboard needs voltages other than +5, this can be accomplished by installing regulators and capacitors in the area near the keyboard connector. For details, you had best consult the factory.

The keyboard must be a fully encoded ASCII keyboard. Either positive true or negative true data will work, provided the switch on the terminal board is set properly. Functions such as N-key rollover and repeat must be handled by the keyboard, since the terminal has no control over them.

The normal condition of bit 7 must be "false" (low for positive true data, high for negative true data) or the keyboard will not work right.

The keyboard strobe can be either positive or negative true, again set by a switch on the board. The hardware description goes into more detail on this.

An optional but desirable feature is a key which turns on bit 7, independent of which other key is being typed (the IKB-1 has such a key, labeled "FLAG I"). The presence of this key allows special local functions to be performed by the number keys at the top of the keyboard (assuming a standard layout). Holding this special key down will activate these functions when the proper number key is pressed:

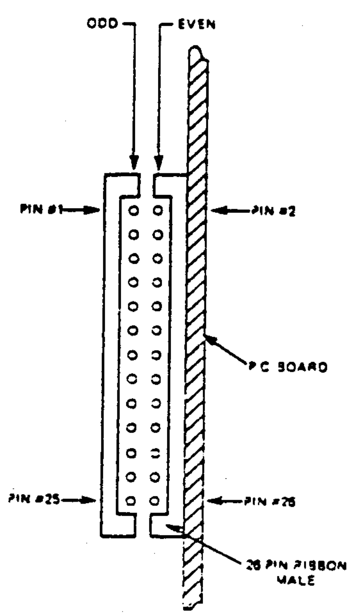
CODE	KEY	FUNCTION
B1H	1	Roll screen down
B2H	2	Roll screen up
B3H	3	Turn 25th line status field on or off
B4H	4	Turn transparent mode on or off
B5H	5	Set line feed delay (see note)
	6-9	Unused
B0H	0	Clear screen

These functions are local, i.e., they do not transmit anything to the host, and they do not require any echo from the host.

Additionally, holding FLAG I and typing CTRL-A (resulting code is 81H), does a soft reset to the terminal, identical to power-on. (If a severe glitch happens and the terminal is "out to lunch", this may or may not work. Its main reason for existence is due to certain edit programs leaving the terminal in a funny mode when they exit).

VIO-X
KEYBOARD INPUT CONNECTOR (J1)

J1 PIN#	DB-25S PIN#	SIGNAL NAME	DESCRIPTION
18	22	DI7	DATA BIT 7
16	21	DI6	DATA BIT 6
14	20	DI5	DATA BIT 5
12	19	DI4	DATA BIT 4
10	18	DI3	DATA BIT 3
08	17	DI2	DATA BIT 2
06	16	DI1	DATA BIT 1
04	15	DI0	DATA BIT 0
22	24	RDY	DATA READY (STROBE)
20	23	RST	RESET
01	01	GND	GROUND
23	12	+5	+5VDC
02	14	+12	+12VDC (OPT)
21	11	SPK	SPEAKER OUT
24	25	-12	-12VDC (OPT)



If your keyboard has special keys for some or all of the above functions, they will function properly iff they emit the code indicated in the table.

Note: The Set Delay function requires that a second character be typed which indicates how long a delay to execute (See ESC D description in the ESCAPE SEQUENCES section). This character must be typed while still holding the FLAG I key down. Also, you should not try to set the line feed delay while characters are coming in from the host, because the first character which happens to show up after the FLAG-I 5 is hit will be interpreted as the delay character.

VIO-X

INTELLIGENT VIDEO INTERFACE

USER MANUAL

REVISION A1.1 08/06/81

FULCRUM™
COMPUTER PRODUCTS
Distributed by:
WW COMPONENT SUPPLY INC. 1771 JUNCTION AVENUE • SAN JOSE, CA 95112 • (408) 295-7171

ESCAPE SEQUENCES

An escape sequence consists of at least two characters, the first being the ESC character (1BH). The second character determines which escape sequence is to be executed. Some escape sequences need more characters as part of their processing, but there is always a minimum of two.

This is a table of all the escape sequences:

Char	Meaning	Action Taken
*	Clear Screen	Clears screen to blanks
+	Clear Screen	Clears screen to blanks
(	Bright att on	Sets active attribute to normal
)	Dim att on	Sets active attribute to dim
,	Status on	Turns on 25th line status field
.	Status off	Turns off 25th line status field
>	Set time	Sets 25th line clock
<	Read time	Reads time from 25th line clock
#	Lock keyboard	Disables keyboard
"	Unlock keyboard	Enables locked keyboard
=	Load cursor	Cursor moves to selected position
?	Read cursor	Cursor position transmitted to host
8	Write 25th line	Writes to 25th line user area
9	25th line att	Sets the 25th line video attribute
D	Set LF delay	Sets delay between line feeds
E	Insert line	Inserts blank line at cursor
H	Cursor off	Turn off cursor
K	Cursor on	Turn on cursor
R	Delete line	Deletes line under cursor
T	Erase line	Clear to end-of-line
U	Set Trans. mode	Turns on Transparent mode
Y	Erase page	Clear to end-of-page
Z	Set video att.	Sets video attribute
DEL	Read version	Transmit firmware version number

Explanation:

a) Clear Screen (ESC + or ESC *)

The terminal responds to ESC + or ESC * by clearing the screen to blanks. The cursor is moved to the home position, and the 24 line window is moved to the top of the 41 line display. The 25th line is not affected.

This function is identical to CTRL-Z.

j) CTRL-[(Escape)

Escape initiates an escape sequence. Escape sequences are explained in detail below.

k) CTRL-^ (Home Cursor)

CTRL-^ causes the cursor to be positioned at the upper left corner of the 41 line display. That is, the 24 line window is moved to the top of the 41 lines, and the cursor is set to the upper left corner of the screen.

l) CTRL-_ (New-Line)

CTRL-_ causes a combined carriage return and line feed. The screen will scroll if the cursor is at the bottom of the screen. The effect is identical to the sequence CTRL-M, CTRL-J.

VIO-X INTRODUCTION

The VIO-X Video I/O Interface for the S-100 bus provides features equal to most intelligent terminals both efficiently and economically. It allows the use of standard keyboards and CRT monitors in conjunction with existing hardware and software. It will operate with no additional overhead in S-100 systems regardless of processor or system speed.

Through the use of the Intel 8275 CRT controller with an onboard 8085 processor and 4k memory, the VIO-X interface operates independently of the host system and communicates via two ports, thus eliminating the need for host memory space. The screen display rate is effectively 80,000 baud.

The VIO-X1 provides an 80 character by 25 line format (24 lines plus status line) using a 5x7 character set in a 7x10 dot matrix to display the full upper and lower case ASCII alphanumeric 96 printable character set (including true descenders) with 32 special characters for escape and control characters. An optional 2732 character generator is available which allows an alternate 7x10 contiguous line and block graphics character set.

The VIO-X2 also offers an 80 character by 25 line format but uses a 7x7 character set in a 9x10 dot matrix allowing high-resolution characters to be used. This model also includes expanded firmware for block mode editing and light pen location. Contiguous graphics characters are not supported.

Both models support a full set of control characters and escape sequences, including controls for video attributes, cursor location and positioning, cursor toggle, and scroll speed. An on-board Real Time Clock (RTC) is displayed in the status line and may be read or set from the host system. A checksum test is performed on power-up on the firmware EPROM.

Video attributes provided by the 8275 in the VIO-X include:

* FLASH CHARACTER
* INVERSE CHARACTER
* UNDERLINE CHARACTER or
* ALT. CHARACTER SET
* DIM CHARACTER

The above functions may be toggled together or separately.

LIGHT PEN NOT SUPPORTED ON X1 FIRMWARE?

The board may be addressed at any port pair in the IEEE 696 (S-100) host system. Status and data ports may be swapped if necessary. Inputs are provided for parallel keyboard and for light pen as well as an output for audio signalling. The interrupt structure is completely compatible with Digital Research's MP/M (TM).

Additional features include:

*HIGH SPEED OPERATION
*PORT MAPPED IEEE S-100 INTERFACE
*FORWARD/REVERSE SCROLL or
*PROTECTED SCREEN FIELDS
*CONVERSATIONAL or BLOCK MODE (opt)
*INTERRUPT OPERATION
*CUSTOM CHARACTER SET
*CONTROL CHARACTERS
*ESCAPE CHARACTER COMMANDS
*INTELLIGENT TERMINAL EMULATION
*TWO PAGE SCREEN MEMORY
*TRANSPARENT MODE

VIO-X Configuration

The VIO-X plugs into any IEEE S-100 bus computer. It contains a keyboard connector, a light pen input, both composite and separate video outputs, and a speaker port.

1) I/O ADDRESS

Before attempting to use the board, its I/O address switches have to be set. They are located on the DIP switch at the bottom of the board. The VIO-X uses two contiguous I/O ports; the lower of the two is always an even-numbered port.

Port address bit 7	xxxxxx	Closed = 1
Port address bit 6	xxxxxx	
Port address bit 5	xxxxxx	
Port address bit 4	xxxxxx	
Port address bit 3	xxxxxx	
Port address bit 2	xxxxxx	
Port address bit 1	xxxxxx	
Status/data swap	xxxxxx	Closed for status = even

To set the address, set the 7 address bits as shown. This addresses the port pair to start at the desired address.

The Status/Data swap switch must also be set: closed for status on even port, data on odd port; open for status on odd port, data on even port.

Here is a typical example: to set the board for SIO compatiblity, the board is set to I/O ports 2 and 3, with data on 2 and status on 3. To do this, the switches are set as follows:

Switch 7 OPEN
Switch 6 OPEN
Switch 5 OPEN
Switch 4 OPEN
Switch 3 OPEN
Switch 2 OPEN
Switch 1 CLOSED
Status/Data swap OPEN

2) STATUS BITS

The bits in the status register must be configured next. The jumper area above and to the right of the Dip switch is used for this purpose. The two pads on the left are:

> Upper pad is Data Ready, which indicates to the host that it may read a character from the terminal;
>
> Lower pad is Transmitter Buffer Empty, which tells the host that it may send a character to the terminal.

The eight pads on the right correspond to bit 7 (top pad) through bit 0 (bottom pad) of the status port. Thus, you connect the Data Ready pad to the status port bit in which the Data Ready bit should appear during a status port read; similarly, the Transmitter buffer empty bit is connected to its appropriate bit.

Here is an example. Once again it provides SIO compatibility.

The SIO uses 8251 USARTs which put the Data Ready bit on bit 1, and the Transmitter Buffer Empty bit on bit 0 of the status port. The Data Ready (upper) pad therefore connects to pad 1 (second from the bottom) and the Transmitter Buffer Empty pad connects to pad 0 (bottom pad).

This completes the configuration process as far as the host interface is concerned.

3) OPTIONS

There are several other parameters which also need to be set:

1) Underline enable/Alternate Character set enable: allows either the Underline or the Alternate Character set feature to be used.

2) Keyboard strobe invert: allows a keyboard with either a positive or negative strobe to be used.

3) Block/Conversational mode startup: Determines the mode in which the terminal powers up. (NOTE: This is only effective when block mode firmware is installed.)

4) Cursor type: Selects between a blinking or non-blinking cursor.

5) Keyboard data invert: allows a keyboard with inverted data to be used.

6) Scroll Delay: This is a three bit value which determines the default delay time between successive line feeds.

Switch Assignments:

Underline enable	xxxxxx	Open = Enabled
Keyboard Strobe Invert	xxxxxx	Closed = Rising edge latch
Conv/Block Initialize	xxxxxx	Open = Conv mode powerup
Cursor Blink/Nonblink	xxxxxx	Open = Non-blink
Keyboard Data Invert	xxxxxx	Closed = Invert Data
Scroll Speed<2>	xxxxxx	See table
Scroll Speed<1>	xxxxxx	See table
Scroll Speed<0>	xxxxxx	See table

Scroll Speed<2:0> cause the terminal to wait a certain amount of time before performing a scroll. This time delay does not affect any cursor manipulation which does not result in a scroll, such as cursor positioning, carriage return, or line feed when the cursor is not at the bottom of the screen.

The delay time is read from the switches at reset time, but can be changed via software, using the ESC D sequence.

The switches set the delay time according to this table:

<2>	<1>	<0>	Resulting Delay (ms)
C	C	C	0.0
C	C	O	33.3
C	O	C	66.7
C	O	O	100.0
O	C	C	133.3
O	C	O	166.7
O	O	C	200.0
O	O	O	233.3

The keyboard strobe invert switch is not needed for keyboards whose data is valid for the entire time the strobe is active. Either switch setting will work equally well (assuming the strobe is a pulse, not a level which remains active the entire time a keyboard key is active). For other keyboards, set the switch as follows:

If the keyboard data is valid at the low-to-high transition of the strobe, set the invert switch to ON.

If the keyboard data is valid at the high-to-low transition of the strobe, set the invert switch to OFF.

For keyboards whose strobe is a level which lasts more than a few milliseconds (including those which stay on until the key is released), set the invert switch as follows:

If the strobe is active high, set the invert switch to ON.

If the strobe is active low, set the invert switch to OFF.

The Conv/Block Initialize switch selects the mode in which the terminal powers up. Normally, the terminal should power up in Conversational mode (switch open), but for applications which require powering up in Block mode, this capability is provided (one application is trying out the keyboard interface without system software, since software is required to set Block mode if the terminal powers up in Conversational mode). Regardless of the setting of the switch, the terminal can be set to the other mode by using the appropriate escape sequence (ESC B OR ESC C) if the block firmware is installed.

The cursor blink/nonblink switch is read at power-on and sets the blink mode of the cursor. If the switch is closed, the cursor blinks. The switch is only read after a reset or power on, so changing the setting of the switch will have no effect until a reset is performed.

The Keyboard Data Invert switch is provided for keyboards (such as the IKB-1) which generate inverted data. When this switch is closed, the keyboard data is inverted by the terminal firmware.

The Underline enable switch is only needed when the alternate character set option is installed. When there is no alternate character set, this switch should be opened to allow the underline attribute to function. If the alternate character set is installed, the switch must be closed; otherwise any alternate character fields will also be underlined (since they are generated by the same video attribute).

NOTE: To use with MPU-B, insure that MPU-B U12 switches 7 and 8 (top two) are in the off position and that the keyboard is plugged into J1 on the VIO-X1.

ON-BOARD JUMPERS

The functions of the several jumpers on the VIO-X board are as follows:

A) CUT TRACE TO REMOVE SPEAKER OUTPUT FROM J1 KEYBOARD CONNECTOR

B) VIDEO TYPE: COMPOSITE - bottom/NON COMPOSITE - top

C) VERTICAL SYNC: POSITIVE=top/NEGATIVE=bottom

D) NOT USED

E) 2716/2732 CHARACTER GENERATOR (CUT TRACE & JUMPER FOR 2732)

F) 2717/2732 FIRMWARE EPROM (CUT TRACE & JUMPER FOR 2732)

G) (NOT MARKED ON REV A BOARDS - LOCATED BELOW THE STATUS JUMPER AREA) VIDEO TYPE: closed=COMPOSITE/open=NON COMPOSITE VIDEO

NOTE:
R19 IS NOT DESIGNATED ON REV A BOARDS - IT IS LOCATED BELOW JUMPER AREA "C"

VIO-X
KEYBOARD INPUT CONNECTOR (J1)

DB-25S PIN#	SIGNAL NAME	DESCRIPTION
22	DI7	DATA BIT 7
21	DI6	DATA BIT 6
20	DI5	DATA BIT 5
19	DI4	DATA BIT 4
18	DI3	DATA BIT 3
17	DI2	DATA BIT 2
16	DI1	DATA BIT 1
15	DI0	DATA BIT 0
24	RDY	DATA READY (STROBE)
23	RST	RESET
01	GND	GROUND
12	+5	+5VDC
14	+12	+12VDC (OPT)
11	SPK	SPEAKER OUT
25	-12	-12VDC (OPT)

VIO-X
CRT VIDEO CONNECTOR (J3)

PIN #	SIGNAL	DESCRIPTION
01	HOR	HORIZONTAL SYNC
02	GND	
03	GND	
04	VER	VERTICAL SYNC
05	GND	
06	VID	VIDEO OUT
07	GND	
08	GND	
09	CVID	COMPOSITE VIDEO
10	N/C	
11	N/C	
12	GND	
13	LPN	LIGHT PEN INPUT

VIO-X Hardware Interface

The VIO-X terminal plugs into the S-100 bus and acts as an I/O device. Because the terminal is an I/O mapped device, the internal RAM is not accessible to the host. All software interaction must therefore take place via two host system I/O ports - status and data.

Status port

The status port can be both written to and read by the host.

1) STATUS READ

In order to determine if the terminal is ready to receive a character from the host, or if it has a keyboard character available, the status port provides two status bits called RxRDY and TxRDY. These bits may appear in any position in the status byte, determined by jumpers on the board. None of the other six bits are used and are read as ones. The TxRDY bit is high whenever the terminal is ready to receive a character. When the host writes a character to the terminal, TxRDY will go low and stay low until the terminal processor has read the character out of its input register. TxRDY will then go high and a new character can be written. The RxRDY bit goes high whenever the terminal has a character to transmit to the host, for whatever reason (could be a keyboard character, a response to a read cursor command, etc). Data read from the terminal data port is valid only when the RxRDY bit is on. The act of reading the data automatically resets the RxRDY bit.

2) STATUS WRITE

In order to set up the VIO-X for interrupt operation (under MP/M for example), it is necessary to enable and disable transmit and receive interrupts in the terminal. This is done by writing to the status port:

Bits 7-4 are unused.
Bit 3 = Enable Receive interrupt
Bit 2 = Disable Receive interrupt
Bit 1 = Enable Transmit interrupt
Bit 0 = Disable Transmit interrupt

If you attempt to both enable and disable an interrupt at the same time, the interrupt enable flipflop will toggle to the opposite state.

The VIO-X must be properly configured with jumpers before interrupts will work properly. This is explained in the Configuration section.

DATA PORT

All characters written to and read from the terminal pass through the data port. When reading data, the terminal sends seven bit characters with the uppermost bit equal to zero. When writing data, the terminal reads seven bit characters from the host, ignoring the upper bit. Here are typical examples of input and output routines. For simplicity, assume SIO 2-2 compatibility:

Data port = 02H
Status port = 03H
TxRDY = bit 0
RxRDY = bit 1

```
;This routine  waits for TxRDY to go high,  and then outputs  the
;character  in  the  B  register  to  the  terminal.
;
DATA      EQU   02H
STATUS    EQU   03H
TXRDY     EQU   01H
RXRDY     EQU   02H
;
CHROUT    IN    STATUS          ;READ  STATUS  PORT
          ANI   TXRDY           ;MASK  OFF  ALL BUT TXRDY BIT
          JZ    CHROUT          ;IF TXRDY LOW, KEEP LOOPING
          MOV   A,B             ;TXRDY HIGH, GET CHAR  FROM B
          OUT   DATA            ;OUTPUT IT
          RET                   ;DONE.
;
;  This routine waits for RxRDY to go high, then reads the character
;  from the terminal and returns with the character in the A register.
;
CHRIN      IN     STATUS        ; READ STATUS PORT
           ANI    RXRDY         ; MASK OFF ALL BUT RXRDY BIT
           JZ     CHRIN         ; LOOP UNTIL RXRDY GOES HIGH
           IN     DATA          ; RXRDY HIGH,  READ THE CHARACTER
           RET                  ; DONE.
```

For a polled single user system (most CP/M systems, for example), these routines are all you need to input and output characters to/from the terminal.

VIO-X Software Interface

There are three different ways for the host to make the terminal do something:

1) Send a normal character.
2) Send a control character.
2a) Send an escape sequence.

1) Normal Characters

There are 96 normal characters, i.e., the entire ASCII character set. Transmitting any of these characters to the terminal results in the character appearing on the screen at the cursor position. The cursor then moves one position to the right. If the cursor is at the right side of the screen when this occurs, it wraps to the left side, moving down one line. If the cursor is at the bottom right corner, it will move to the bottom left corner and the screen will scroll up, leaving a blank line at the bottom.

2) Control characters

There are 32 control characters in the ASCII character set. The terminal responds to some (but not all) of them. The characters which the terminal notices are:

Char	Hex	Meaning	Action Taken
@	00H	NULL	Ignored by terminal ***
D	04H	ROLL DOWN	Scrolls screen down one line
G	07H	BELL	Beeps Beeper
H	08H	BACKSPACE	Bumps cursor to the left
J	0AH	LINE FEED	Bumps cursor down one line
K	0BH	CURSOR UP	Bumps cursor up one line
L	0CH	CURSOR RIGHT	Bumps cursor to the right
M	0DH	CAR RETURN	Returns cursor to left side
U	15H	ROLL UP	Scrolls screen up one line
Z	1AH	CLEAR SCRN	Clears the screen to blanks
[	1BH	ESCAPE	Initiates an escape sequence

While the terminal does not respond to any other control characters, you can generate any of them by holding down CTRL and hitting the appropriate key. The terminal will then transmit that control character to the host.

*** The NULL (00H) character is required by most serial terminals to provide a delay for screen clearing and processing various other special functions. While the VIO-X does not mind receiving nulls, they are UNNECESSARY because, unlike a serial terminal, the VIO-X hardware provides full handshaking via the TxRDY and RxRDY status bits. Therefore, for maximum speed, you never need to insert nulls after any command.

Explanation of control characters:

a) CTRL-@ (NULL)

The terminal does not respond to null characters (see above). However, the terminal will recognize a null rather than search its entire control character table, in order to provide maximum speed using software which sends nulls.

b) CTRL-D (Roll down) and CTRL-U (Roll up)

These functions make use of the 49 line display feature. Only 24 lines can be displayed at a time, so some way to look at the hidden lines is needed. This is accomplished with CTRL-D and CTRL-U.

The screen is like a "window" of 24 lines which can be moved up and down the 49 line screen. When the window moves up, the screen appears to roll down, and vice versa.

When the screen is cleared, this 24 line window is positioned at the top of the 49 line display. As the screen fills with characters, the window stays at the top until the cursor reaches the bottom of the screen and a scroll occurs. The window then moves down one line, scrolling the screen up one line.

This process continues until all 49 lines are full. The window is now at the bottom of the 49 line display. Since the window cannot move down any more, the top line is cleared to blanks (you cannot see this line since the window is at the bottom) and the entire 49 line display is rolled up. This blanked line then appears at the bottom of the screen. As long as the window is at the bottom, scrolling is done this way.

This means that up to 49 lines of text can be saved in the terminal memory before the oldest lines start to disappear. The host can move the window up (CTRL-D) or down (CTRL-U) to display lines which have scrolled off the top of the screen. Thus CTRL-D scrolls the screen down, while CTRL-U scrolls it up.

Note that the window cannot be moved past the boundaries of the 49 lines, so rollup/rolldown stops when the boundary is reached.

c) CTRL-G (Bell)

When the terminal receives CTRL-G, it beeps the audible beeper, if installed.

d) CTRL-H (Backspace)

CTRL-H bumps the cursor one space to the left. If the cursor is already at the left side of the screen, it wraps around to the right side, moving up one line. If it is in the top left corner of the screen, it does not move.

e) CTRL-J (Line Feed)

CTRL-J moves the cursor down one line if it is not already at the bottom of the screen. If it is at the bottom, it stays there; but the screen scrolls up one line, leaving a blank line at the bottom.

f) CTRL-K (Cursor Up)

CTRL-K moves the cursor up one line if it is not already at the top of the screen. If it is at the top, it stays there; and if the window is not at the top of the 49 line display, the screen rolls down one line. (If the window is at the top, CTRL-K does nothing).

g) CTRL-L (Cursor Right)

CTRL-L moves the cursor one space to the right, wrapping and scrolling as needed. It differs from <space> in that it does not write a blank to the screen, but only moves the cursor.

h) CTRL-M (Carriage Return)

CTRL-M moves the cursor to the left side of the current line.

i) CTRL-Z (Clear Screen)

CTRL-Z clears the entire 49 line display to blanks and positions the 24 line window at the top. The 25th line is not affected. This funtion is identical to ESC + (see Escape Sequences).

j) CTRL-[(Escape)

Escape initiates an escape sequence. Escape sequences are explained in detail as follows:

ESCAPE SEQUENCES

An escape sequence consists of at least two characters, the first being the ESC character (1BH). The second character determines which escape sequence is to be executed. Some escape sequences need more characters as part of their processing, but there is always a minimum of two. This is a table of all the escape sequences:

Char	Meaning	Action Taken
+	Clear Screen	Clears screen to blanks
>	Set time	Sets 25th line clock
<	Read time	Reads time from 25th line clock
#	Lock keyboard	Disables keyboard
"	Unlock keyboard	Enables locked keyboard
=	Load cursor	Cursor moves to selected position
?	Read cursor	Cursor position transmitted to host
~~4~~	Send line	Transmit current line to host
~~5~~	Send page	Transmit entire screen
8	Write 25th line	Writes to 25th line user area
D	Set LF delay	Sets delay between line feeds
H	Cursor off	Turn off cursor
K	Cursor on	Turn on cursor
T	Line Erase	Erase from cursor to end of line
U	Set Trans. mode	Turns on Transparent mode
Y	Page Erase	Erase from cursor to end of page
Z	Set video att.	Sets video attribute
DEL	Read version	Transmit firmware version number

Deleted (4 5)

See Rev A1.1

Explanation:

a) Clear Screen (ESC +)

The terminal responds to ESC + by clearing the screen to blanks. The cursor is moved to the home position, and the 24 line window is moved to the top of the 49 line display. The 25th line is not affected. This function is identical to CTRL-Z.

b) Set Time (ESC >)

This function allows you to set the 25th line clock. Since the clock display consists of six digits, six characters are required to set the clock. These should be in the range "0" (30H) through "9" (39H) and the sequence of transmitting to the terminal is:

Hours tens, hours units, ..., seconds units

The clock only knows about 24 hour (military) time. Also, the internal counter is reset when the time is set, so the time to the first count is always one second from the transmission of the last character in the sequence (the time does not appear on the 25th line until the last character has been sent).

For example, to set the clock to 10:33:46, the following sequence is transmitted:

ESC > 1 0 3 3 4 6

The clock does not check for illegal values and it is quite possible to put anything into the clock display if you're not careful.

If any of the six characters is "@", the clock display is blanked and will remain blanked until the clock is set again. Furthermore, the sequence aborts as soon as the "@" is received, so that subsequent characters will appear on the screen at the cursor location.

c) Read time (ESC <)

When it receives ESC <, the terminal transmits the contents of the clock as a sequence of six characters. The format is the same as for setting the clock.A carriage return (0DH) is sent to terminate the sequence.

Note: If the clock has not been set, or has been set and subsequently shut off via ESC > @, then ESC < will transmit six ASCII space characters, followed by <cr>.

d) Lock Keyboard (ESC #)

When the host wishes to disable the terminal's keyboard for any reason, it can do so by transmitting ESC #. The terminal will then lock the keyboard and display KBD LOCK in inverse bright characters on the 25th line. In addition, the cursor is shut off. The keyboard will remain locked until the system unlocks it, or the user types CTRL-@.

e) Unlock Keyboard (ESC ")

The host transmits ESC " to unlock the keyboard. The KBD LOCK message goes away and the keyboard is once again functional. The cursor also re-appears, unless it has been disabled via ESC H.

f) Load Cursor (ESC =)

The host can position the cursor anywhere on the screen by transmitting this sequence. The complete sequence is ESC = yx, where y is the row and x is the column in which the cursor is to sit. Row or column zero is selected by space; other values are calculated by adding 20H to the row or column number. For example, the lower right corner of the screen (row 23, column 79) is selected by ESC = 7o. (See table below.)

ABSOLUTE
CURSOR POSITIONING

x/y	ASCII CODE	x/y	ASCII CODE	x/y	ASCII CODE
1	space	28	;	55	V
2	!	29	<	56	W
3	"	30	=	57	X
4	#	31	>	58	Y
5	$	32	?	59	Z
6	%	33	@	60	[
7	&	34	A	61	\
8	'	35	B	62	]
9	(	36	C	63	^
10	)	37	D	64	_
11	*	38	E	65	`
12	+	39	F	66	a
13	,	40	G	67	b
14	-	41	H	68	c
15	.	42	I	69	d
16	/	43	J	70	e
17	0	44	K	71	f
18	1	45	L	72	g
19	2	46	M	73	h
20	3	47	N	74	i
21	4	48	O	75	j
22	5	49	P	76	k
23	6	50	Q	77	l
24	7	51	R	78	m
25	8	52	S	79	n
26	9	53	T	80	o
27	:	55	U		

g) Read Cursor Position (ESC ?)

The host can determine the position of the cursor by transmitting ESC ? to the terminal. The terminal responds by sending the sequence yx<cr>, where y is the row (calculated as above), x is the column and <cr> is a carriage return character.

h) Send Line (ESC 4)

When the terminal receives ESC 4, it responds by transmitting the current line (the one on which the cursor sits) to the host. All characters from the left side to the cursor (inclusive) are transmitted. If a video attribute exists in the line, the escape sequence which generated it is transmitted in its place, i.e. ESC Z a. A carriage return character is transmitted at the end of the sequence.

i) Send Page (ESC 5)

Send page is similar to Send Line. The terminal responds to ESC 5 by transmitting to the host all characters on the screen from the home position down to the cursor (inclusive). At the end of each line transmitted, a new-line character (1FH) is transmitted to indicate that a complete line has just been sent. Video attributes on the screen are sent as ESC Z a (exactly like Send Line).

The sequence ends with a carriage return character.

j) Write to 25th line (ESC 8)

The terminal responds to ESC 8 by clearing the 25th line of the display. All subsequent incoming characters are the written to the 25th line instead of the cursor position (the cursor does not move during this time). All normal characters are allowed, as are video attributes. The terminal will continue to put incoming characters on the 25th line until:

1) Any control character other than ESC is received;
2) Any escape sequence other than ESC Z is received;
3) 58 characters (including video attributes) are written.

Writing to the 25th line is normally terminated by a carriage return character, although any of the conditions listed above will do it. Once the termination occurs, operation returns to normal.

To clear the 25th line, send ESC 8 <cr>.

k) Set Line Feed delay (ESC D)

For applications where the scroll rate is too fast to be easily readable, . this sequence will provide a delay which the terminal will wait before it executes a scroll. The complete sequence is ESC D d, where d is a character in the range SPACE to "?". This character tells the terminal how many 16.7 ms intervals (i.e., frames) to wait before it will execute a scroll. The character assignment is as follows:

Char	Frames	Max LF/sec	Char	Frames	Max LF/sec
SPACE	0	Very, very many	0	16	3.8
!	1	60	1	17	3.5
"	2	30	2	18	3.3
#	3	20	3	19	3.2
$	4	15	4	20	3.0
%	5	12	5	21	2.9
&	6	10	6	22	2.7
'	7	8.6	7	23	2.6
(	8	7.5	8	24	2.5
)	9	6.7	9	25	2.4
*	10	6.0	:	26	2.3
+	11	5.5	;	27	2.2
,	12	5.0	<	28	2.1
-	13	4.6	=	29	2.1
.	14	4.3	>	30	2.0
/	15	4.0	?	31	1.9

Note that this delay ONLY affects actual scrolling of the screen. It does not affect anything else; in particular, cursor positioning and line feeds which do not result in a scroll. Roll up and roll down are also unaffected.

l) Disable cursor (ESC H)

When the terminal receives ESC H, it turns off the white square which represents the cursor. It does not affect the cursor in any other way; all cursor manipulation functions still operate and the keyboard is not locked.

m) Enable cursor (ESC K)

This does just the opposite of ESC H, by turning on the white square which represents the cursor. If the keyboard is locked when this is done, the cursor does not come back (remember, the cursor disappears during keyboard lock) until the keyboard is unlocked.

n) Line Erase (ESC T)

Clears the display from cursor to end of current line.

o) Set Transparent Mode (ESC U)

When the terminal receives ESC U, it reacts by setting Transparent mode, which is indicated by the message XPARENT in bright inverse characters on the 25th line. In this mode, all incoming characters, control or not, are displayed on the screen. No control characters cause any special action (one exception -- see below); i.e., carriage return does not perform the normal carriage return function, but displays a special character on the screen instead.

Escape sequences are handled somewhat differently, in that the ESC is not displayed until the next character arrives. The reason for doing this is that there is one escape sequence which is recognized in the transparent mode, namely ESC X, which resets transparent mode.

p) Normal Mode Reset (ESC X)

Resets terminal to normal mode per above from transparent mode.

q) Page Erase (ESC Y)

Clears the display from cursor to end of currently displayed page.

r) Set Video Attribute (ESC Z)

ESC Z causes the terminal to treat the next incoming character as a video attribute code. The video attribute character is converted to a special code and stored at the cursor position (the cursor then bumps over in the usual way). The video hardware causes the attribute character to be displayed as a blank.

As the video logic scans down the screen, it notices any video attributes and activates the appropriate functions, which stay activated until a new video attribute is encountered, or the end of the screen is reached. If, for example, an inverse video attribute exists somewhere on the screen, then all characters between that character and the end of the screen (or the next attribute, if there are any others) will be inverse video. By writing attribute characters in this way, the host system can control the appearance of the screen.

The video attribute as sent by the host is a character in the range " " (space) to "/" (20H to 2FH). The bit assignments are as follows:

Char	Hex	Resulting Attribute
space	20H	Normal
!	21H	Dim
"	22H	Blinking
#	23H	Blinking dim
$	24H	Inverse
%	25H	Inverse dim
&	26H	Inverse blinking
'	27H	Inverse blinking dim
(	28H	Underline (or Alt Char Set)
)	29H	Underline dim
*	2AH	Underline blinking
+	2BH	Underline blinking dim
,	2CH	Underline inverse
-	2DH	Underline inverse dim
.	2EH	Underline inverse blinking
/	2FH	Underline inverse blinking dim

Thus, to write an inverse video attribute, the host sends ESC Z $ ("$" is 24H, the code for an inverse video attribute).

s) Read Version (ESC DEL)

When the terminal receives ESC DEL (7FH), it returns a two character sequence which represents the version number of the firmware PROM currently installed in the terminal. The first character is a letter A-Z and indicates the microcode type. The second character is a digit 0-9 and indicates the revision level of the microcode. A carriage return character is sent to terminate the transmission.

NOTE THAT VIDEO ATTRIBUTES TAKE UP A BLANK DISPLAY POSITION

INTERRUPT OPERATION

The VIO-X contains two flip-flops called TxINT ENABLE and RxINT ENABLE. When the TxINT ENABLE flip-flop is on, the terminal will assert an interrupt whenever it is ready to receive a character. Similarly, when RxINT ENABLE is on, the terminal will interrupt whenever it has a character to transmit to the host.

These flipflops (particularly TxINT ENABLE) are necessary for proper interrupt operation, since it must be possible to disable interrupts sometimes. For example, if the TxRDY interrupt were not disable-able, then a constant interrupt would be asserted whenever characters were not being sent to the terminal.

At power-on and after a reset, the RxINT flipflop is enabled, and the TxRDY flipflop is disabled. The software can change the settings of these flipflops by writing to the status port:

Bits 7-4 are unused.
Bit 3 = Enable RxINT
Bit 2 = Disable RxINT
Bit 1 = Enable TxINT
Bit 0 = Disable TxINT

Note that the settings of these flags are not readable by the software.

The RxINT ENABLE flipflop is normally left in the enabled state, since, in the idle state, the terminal has nothing to send and the interrupt will not be asserted.

The TxINT ENABLE flipflop is normally left in the disabled state, since, in the idle state, the terminal is ready to receive a character and the interrupt would therefore be asserted all the time.

This configuration matches the power-up/reset configuration of the two flipflops.

Here is an example of interrupt driven I/O. It is not as simple as the example above, but it allows terminal I/O to occur while other processes are taking place. The following assumptions apply:

The Receive interrupt routine resides at 0000H (RST 0)
The Transmit interrupt routine resides at 0008H (RST 1)
The terminal configuration is the same as in the above example.

```
;
; Receiver Interrupt sends us here, jump to where we have more room.
;
RST0:     ORG    0000H
          JMP    RXINT
;
; Transmitter Interrupt sends us here, jump to more roomy place
;
RST1:     ORG    0008H
          JMP    TXINT
                 .
                 .
                 .
                 .
;
; Receiver Interrupt routine
;
RXINT     PUSH   PSW              ; SAVE ALL CLOBBERED REGISTERS
          PUSH   B                ;
          PUSH   H                ;
          LHLD   INPOINT          ; GET INPUT POINTER
          IN     DATA             ; INPUT DATA FROM TERMINAL
          MOV    B,A              ; WRITE TO BUFFER
          LDA    TERMCHR          ; GET TERMINATION CHARACTER
          CMP    B                ; COMPARE WITH CHAR JUST READ
          JZ     FULL             ; IF IT MATCHES, TERMINATE READ
          MOV    M,B              ; DOESN'T MATCH, STORE CHAR IN BUFFER
          INX    H                ; INCREMENT POINTER BUFFER
          SHLD   INPOINT          ; SAVE IT FOR NEXT TIME
          LDA    CNTREAD          ; GET CHARACTER COUNT
          INR    A                ; INCREMENT IT
          STA    CNTREAD          ; STORE IT AGAIN
          LDA    READCNT          ; GET INPUT COUNTER
          DCR    A                ; DECREMENT IT
          STA    READCNT          ; STORE AWAY FOR NEXT TIME
          JZ     FULL             ; IF ZERO, NO MORE CHARS TO READ
RXDONE    POP    H                ; RESTORE CLOBBERED REGISTERS
          POP    B                ;
          POP    PSW              ;
          EI                      ; DON'T FORGET!!
          RET                     ; DONE.
```

```
FULL    MVI   A,04H         ; WE GET HERE WHEN DONE WITH READ
        OUT   STATUS        ; TURN OFF RXINT ENABLE FLIPFLOP
        MVI   A,OFFH        ;
        STA   RXCMPLT       ; SET READ COMPLETED FLAG
        JMP   RXDONE        ; REALLY DONE.
;
; Transmitter Interrupt Routine
;
TXINT   PUSH  PSW           ; SAVE ALL CLOBBERED REGISTERS
        PUSH  H             ;
        LDA   OUTCNT        ; GET OUTPUT COUNT
        ORA   A             ; CHECK IF EQUAL TO ZERO
        JZ    EMPTY         ; IF SO, DO NOT TRANSMIT
        DCR   A             ; NOT ZERO, DECREMENT OUTPUT COUNTER
        STA   OUTCNT        ; STORE AWAY FOR NEXT TIME
        LHLD  OUTPNT        ; GET OUTPUT BUFFER POINTER
        MOV   A,M           ; LOAD CHARACTER FROM BUFFER
        OUT   DATA          ; OUTPUT TO TERMINAL
        INX   H             ; INCREMENT BUFFER POINTER
        SHLD  OUTPNT        ; STORE AWAY FOR NEXT TIME
TXDONE  POP   H             ; RESTORE CLOBBERED REGISTERS
        POP   PSW           ;
        EI                  ; RE-ENABLE INTERRUPTS
        RET                 ; DONE.
EMPTY   MVI   A,01H         ; NO MORE CHARACTERS IN BUFFER
        OUT   STATUS        ; TURN OFF VIO-X TXINT ENABLE FLIPFLOP
        MVI   A,OFFH        ; SET WRITE COMPLETED FLAG
        STA   TXCMPLT       ;
        JMP   TXDONE        ; REALLY DONE.
```

The RXINT routine is invoked whenever a character is ready from the terminal (assuming RXINT ENABLE is set). The characters are read and fill a memory buffer (pointed to by INPOINT). The number of characters read is contained in CNTREAD.

If a character is read which matches the termination character TERMCHAR, the read is terminated and the RXCMPLT flag is set to let the software know. A typical termination character would be <cr>. When the read terminates, the RxINT ENABLE flipflop is reset to prevent any more characters from coming in, until the next buffer is readied. In addition, the flag RXCMPLT is set to inform the software that the read has been completed.

If a termination character never shows up, the buffer fills until the maximum number of characters (as specified by READCNT at the time the read is initiated) have been read. The read then terminates as above.

The TXINT routine is invoked whenever a the terminal is ready to receive a character (assuming the TXINT ENABLE flipflop is set, of course). The routine empties a buffer one character at a time, until the location OUTCNT is exhausted. When this happens, the TxINT ENABLE flipflop is reset and the flag TXCMPLT is set to inform the software that the write has finished.

This routine sets up an input buffer of 80 characters. The termination character is set to <cr>. For this example, the routine loops waiting for RXCMPLT to go true, although a real application could be doing other things during this time.

```
;
; Buffer Read routine
;
BUFRD     LXI    H,BUFFER      ; CHARACTERS WILL SHOW UP HERE
          MVI    A,80          ; READ 80 CHARACTERS MAX
          STA    READCNT       ;
          XRA    A             ; INIT. COUNT READ TO ZERO
          STA    CNTREAD       ;
          STA    RXCMPLT       ; CLEAR READ COMPLETED FLAG
          MVI    A,0DH         ; CARRIAGE RETURN CHARACTER
          STA    TERMCHAR      ; SET TERMINATION CHARACTER
          MVI    A,08H         ; ENABLE RXINT ENABLE
          OUT    STATUS        ;
BUFRDL1   LDA    RXCMPLT       ; WAIT FOR READ COMPLETE FLAG TO GO TRUE
          ORA    A             ;
          JZ     BUFRDL1       ;
                 .
                 .             ; READ COMPLETE, GO PROCESS BUFFER...
                 .             ; COUNT READ WILL BE IN CNTREAD.
```

The following routine takes a buffer BUFFER of 80 characters and outputs it to the terminal. The number of characters in the buffer is stored in OUTCNT, and the pointer to buffer is OUTPNT. Again, the software loops until the write is finished, although this is not the typical case.

```
;
; Buffer Write routine
;
BUFWRT    LXI    H,BUFFER      ; POINT AT OUTPUT BUFFER
          SHLD   OUTPNT        ; STORE INTO OUTPUT POINTER
          MVI    A,80          ; 80 CHARACTERS TO WRITE
          SHLD   OUTCNT        ; STORE INTO OUTPUT COUNTER
          XRA    A             ;
          STA    TXCMPLT       ; CLEAR TRANSMIT COMPLETE FLAG
          MVI    A,02H         ;
          OUT    STATUS        ; SET TXINT ENABLE FLAG
```

```
BUFWRL1  LDA   TXCMPLT      ; LOOP UNTIL TXCMPLT GOES TRUE.
         ORA   A            ;
         JZ    BUFWRL1      ;
               .
               .            ; WRITE COMPLETE, GO ON TO OTHER THINGS
               .
```

Interrupts provide a means of doing I/O while other processes (even other I/O, polled or interrupt driven) are going on. All that is required is to set up the buffer and start the process going; the interrupt routines will take care of the rest. The routines shown here can, of course, be improved to handle more complicated functions, such as backspace; the user will no doubt want to write his own routines to match his requirements.

Since the VIO-X is a single board product, FULCRUM has no control over what type of keyboard is selected for use with it. However, in order to function properly the keyboard must meet certain requirements.

Naturally, its connector must match the VIO-X pinout (refer to the hardware description and schematic). The FULCRUM IKB-1 is compatible with this pinout.

The keyboard must be a fully encoded ASCII keyboard. Either positive true or negative true data will work, provided the switch on the VIO-X terminal is set properly. Functions such as N-key rollover and repeat must be handled by the keyboard, since the terminal has no ability to do this.

The keyboard strobe can be either positive or negative true, again set by a switch on the board. The hardware description goes into more detail on this.

The normal condition of bit 7 must be "false" (low for positive true data, high for negative true data) or the keyboard will not work properly. This bit is used as a special function indicator.

An optional but desirable feature is a key which turns on bit 7, independent of which other key is being typed (the IKB-1 has such a key, labeled "FLG I"). The presence of this key allows special local functions to be performed by the number keys at the top of the keyboard (assuming a standard layout). Holding this special key down will activate these functions when the proper number key is pressed:

CODE	KEY	FUNCTION
B1H	1	Roll screen down
B2H	2	Roll screen up
B3H	3	Turn 25th line status field on or off
B4H	4	Turn transparent mode on or off
B5H	5	Set line feed delay (see note)
	6-9	Unused
B0H	0	Clear screen

These functions are local, i.e., they do not transmit anything to the host, and they do not require any echo from the host.

If your keyboard has special keys for some or all of the above functions, they will function properly if they emit the code indicated in the table.

Note: The Set Delay function requires that a second character be typed which indicates how long a delay to execute (See ESC D description in the ESCAPE SEQUENCES section). This character must be typed while holding the FLG I key down. Also, you should not try to set the line feed delay while characters are coming in from the host, because the first character which happens to show up after the FLG-I 5 is hit will be interpreted as the delay character.

HARDWARE

The advent of CRT controllers in an LSI package has allowed the development of an intelligent video interface to the S-100 bus. The Intel 8275 CRT controller chip was selected because of its features and for compatibility to the 8085 processor. Port mapped I/O was chosen over memory mapped I/O to decrease the system overhead and allow interfacing with virtually any type of system.

The design of the VIO-X is based on the single chip INTEL 8275 CRT Controller which is a programmable NMOS-LSI device. It provides display row buffering, raster timing, cursor timing, light pen detection and visual attribute decoding. It can generate a screen format size of from 1 to 80 characters per row, 1 to 64 rows per screen and 1 to 16 lines per character row.

For compatibility, speed, and ease of implementation, the INTEL 8085 microprocessor was chosen for the on-board processor. The processor clock is derived from dividing the dot clock by 2. The 8085's ability to directly interface with memory and other system components thus minimizing chip count was an important factor in its selection.

To eliminate the need for a DMA controller and at the same time use the feature provided in the 8275 for rapid data transfer, a "trick" was used to simulate the function. The SOD (Serial Output Data) line from the 8085 is used as a toggle to place the 8275 on the data bus in parallel with memory and 40 POP's are done with the display memory reading and the 8275 writing to its buffers simultaneously. The POP moves two bytes from memory to the bus and also from the bus to the 8275 buffer (when SOD is toggled on).

The character generator circuit is similar to those recommended by Intel in AP-32 and AP-62 with the addition of a capability for 2716/2732 EPROM for the character generator. A custom character set was designed for this system which displays full upper and lower case ASCII alphanumeric characters and special control characters.

The video logic section is totally digital (no one-shots) to insure a stable display. Counters are used throughout to generate horizontal and vertical blanking and sync. Both composite and separate video signals are provided as outputs.

The 8275 supports the light pen feature and provision has been made for buffered light pen input to the board. Pen position may be read through an escape command sequence.

Software not provided on X1

The on-board I/O ports are internally memory mapped as follows:

ADDRESS	PORT
7000h	KEYBOARD DATA
6000h	STATUS
5000h	S-100 OUTPUT PORT
4000h	BELL OUTPUT
3000h	S-100 INPUT PORT
2000h	8275 SELECT
1XXXh	DISPLAY MEMORY
0XXXh	FIRMWARE ROM

The keyboard port (7000h) is a latch internally memory mapped to the on-board processor with status output to allow polling. The input data strobe will support both negative and positive logic. The data lines may be either positive or negative logic with the processor firmware providing inversion if required. The keyboard connects via a 26 pin header allowing use of a DB-25 type panel connector. Both +5VDC and -12VDC are provided on the connector.

The board status port (6000h) is used for reading 6 DIP switches (system function inputs), keyboard data available, and host S-100 data available. The function inputs define the mode of operation for the board, cursor type, scroll delay, and keyboard data polarity. All are power up/reset conditions some of which may be changed by command.

The S-100 output port is a latch buffer clocked from the output port select (5000h) and the S-100 port read for the port selected by the board port address switches. The status read function operates in a similar manner. The status and I/O port addresses may be swapped (even/odd) by closing an additional DIP switch.

The S-100 input port works in a similar way from input port select (3000h) and the S-100 port write for the port selected by the board port address switches.

Interrupts are available to the host S-100 system on two maskable outputs (Output Buffer Ready and Input Buffer Ready) and one non-maskable output (Data Ready (to host)). These may be strapped to any of the 8 vectored interrupts on the host S-100 bus or to the priority interrupt bus line.

To eliminate one of the problems encountered with video-keyboard systems, an additional on-board port (4000h) has been defined to allow audio signalling to the user similar to that on a terminal or Teletype input device. This signal is activated when either an ASCII BEL signal is received from the host S-100 system or on command of the on-board processor firmware.

The display memory is on-board and independent of the host system memory. A total of 4096 bytes in four banks is provided by eight 2114 type static memory devices permitting two pages of screen data to be stored. This allows forward and reverse scrolling or screen protected fields to be offered.

FIRMWARE

The firmware is contained in either a 2716 (2k) or 2732 (4k) type EPROM. The version currently offered emulates the SOROC IQ-120 with several added functions. A small change in the firmware enables the board to respond to escape and control command sequences in any way required. The expanded firmware will operate in either conversational or block mode. In conversational mode, the board runs as a full duplex terminal with all normal characters (non-command) passing transparently through the processor to the host and the host providing character echo to the screen.

Alternately, in block mode, the keyboard does not transmit to the host but rather to the screen memory, thus permitting on-board editing without involving the host system. When the data is correct, it is transmitted to the host by a block "send" command of either a line, field, or complete screen.

Protected fields are provided at the cost of reverse scrolling, as the "shadow" screen memory (where protected status flags are kept) is the second page.

A 24 hour Real Time Clock is included as part of the firmware which shows on the 25th line (status line) and is readable and setable fron the host system.

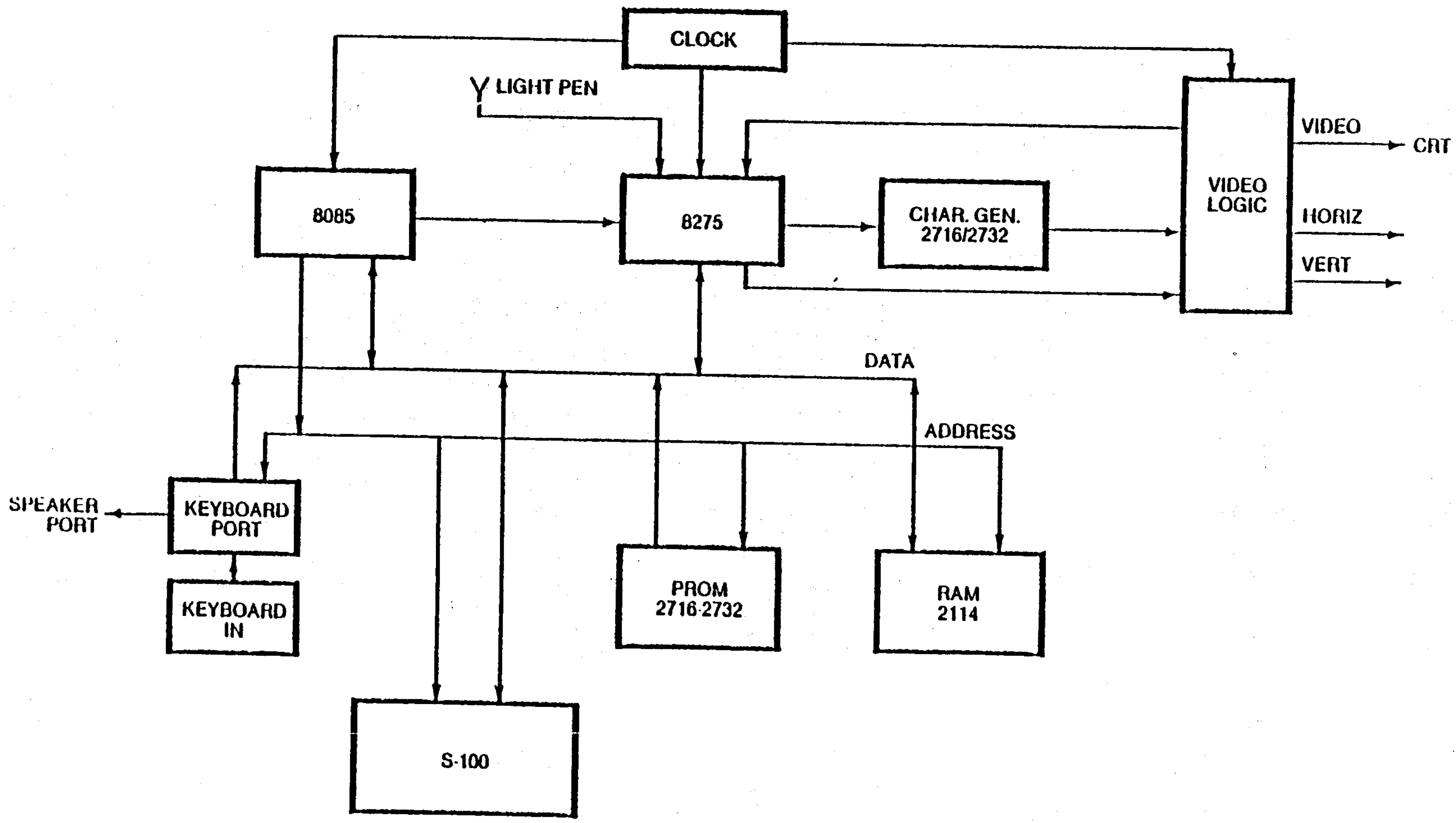
An automatic checksum test is performed on the Firmware PROM on each reset or power up. If an error is detected and the terminal is at all functional, the error code will be displayed in bright inverse on the 25th line.

One unusual feature was added when design tests indicated the hardware/firmware were very efficient in terms of speed. It became necessary to control the scroll rate as the board was as fast or faster that the operating systems it was connected to. When using the TYPE command in CP/M (R) or the L(IST) command in DDT (under CP/M) it was impossible to read the display due to the scroll rate, and difficult to stop the screen anywhere near the area desired. A scroll speed escape sequence command is now used to determine how fast a scroll will occur. This does not affect cursor addressable positioning or screen/ line clears, which still operate at the maximum rate.

REFERENCES

INTEL Application Note AP-32:
"CRT Terminal Design Using the INTEL(R) 8275 and 8279" by John Murray and George Alexy

INTEL Application Note AP-62:
"A Low Cost CRT Terminal Using the 8275" by John Katausky



VIO-X S-100 I/O INTERFACE

Part Number	Description/Designator(s)
30-3470362	RES 470 OHM 1/4W 5% CF R-9 R-16
30-3510362	RES 510 Ohm 1/4W 5% R-5
30-3680362	RES 680 Ohm 1/4W 5% R-13 R-12 R-4
30-3910362	RES 910 Ohm 1/4W 5% R-1 R-3 R-2
30-4100362	RES 1KOHM 1/4W 5% CF R-14 R-11 R-10 R-8 R-18 R-17 R-15
30-4120362	RES 1.2K OHM 1/4 WATT/BROWN RED RED R-7
30-4470362	RES 4.7K 1/4W 5% CF R-19
32-0100331	CAP 100PF 5% SILVER MICA C-16
32-0222020	CAP 22pF Disk Ceramic C-15
32-2010010	CAP .1UF 30V DISC CAP C-34 C-38 C-37 C-36 C-35 C-33 C-32 C-27 C-26 C-25 C-24 C-23 C-22 C-21 C-18 C-17 C-14 C-13 C-12 C-9 C-8 C-7 C-6 C-5 C-4 C-3
32-2202270	CAP 2.2uF TANT 25V C-29 C-28 C-20 C-19 C-31 C-30
35-1000007	DIO 1N4001 TO 1N4007 CR-1
35-2000002	XTR 2N3904 Q-1
35-5000017	XTL 12.2472 MHZ HC18 SERIES RES .02% Y-1
35-6000008	RES 9x1K Ohm Sip RP-1

Part Number	Description/Designator(s)
36-0089701	IC 8T97 8097 74367 U-32
36-0211401	IC 2114 U-13 U-12 U-19 U-18 U-17 U-16 U-15 U-14
36-0740002	IC 74LS00 QUAD 2 INPUT NAND U-42
36-0740401	IC 7404 HEX INVERTER U-41
36-0740402	IC 74LS04 U-30
36-0740502	IC 74LS05 HEX INVERTER OPEN COLLECTOR U-8
36-0740701	IC 7407 U-41
36-0740802	IC 74LS08 QUAD 2 INPUT AND U-36 U-4
36-0741002	IC 74LS10 TRIPLE 3 INPUT NAND U-7
36-0743202	IC 74LS32 U-5 U-35
36-0747401	IC 7474 DUAL D FF U-33
36-0747402	IC 74LS74 U-21
36-0748602	IC 74LS86 QUAD EXOR U-6
36-0780501	IC 7805 TO-220 U-28 U-39 U-20
36-0808501	IC 8085 MICROPROCESSOR CHIP U-49

Part Number	Description/Designator(s)
36-0827501	IC INTEL 8275 CRT CONTROLLER U-11
36-7410902	IC 74LS109 DUAL JK FLIP FLOP U-23
36-7411202	IC 74LS112 U-34
36-7413602	IC 74LS136 U-44 U-45
36-7413802	IC 74LS138 U-51
36-7413902	IC 74LS139 DUAL 2 TO 4 LINE DECODER U-38 U-31
36-7416102	IC 74LS161 SYNCHRONOUS 4-BIT COUNTER U-29
36-7416302	IC 74LS163 SYNCRONUS 4 BIT COUNTER U-24 U-22
36-7416601	IC 74166 U-10
36-7417402	IC 74LS174 HEX LATCH U-9
36-7424102	IC 74LS241 U-47
36-7424402	IC 74LS244 8-BIT BUFFER U-2
36-7437302	IC 74LS373 8-BIT FALL THROUGH LATCH U-50
36-7437402	IC 74LS374 U-48 U-46 U-1

VIO-X
INTERRUPT STRAPPING LIST

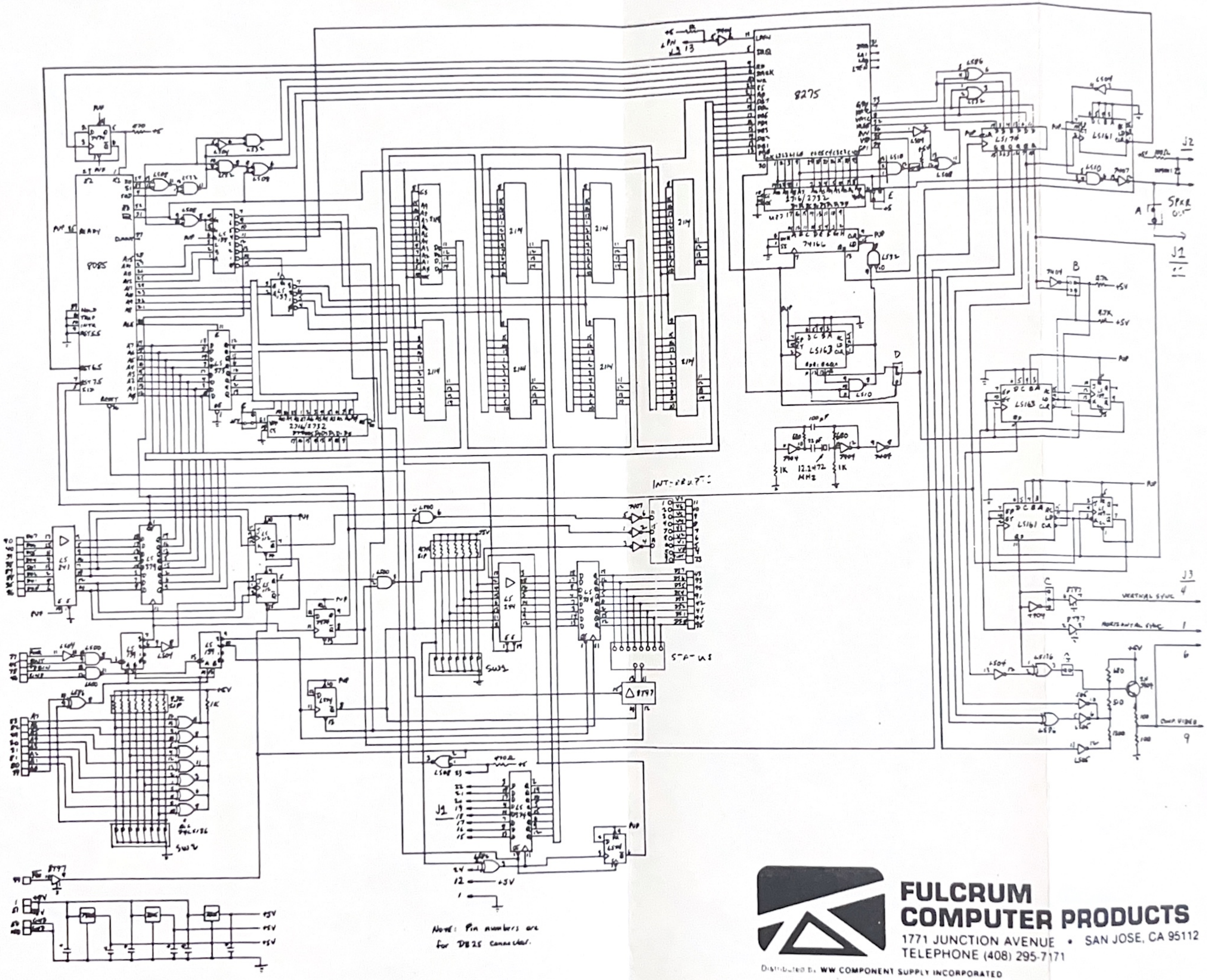
PIN#	SIGNAL	DESCRIPTION
01	V7	VECTORED INTERRUPT 7
02	V6	VECTORED INTERRUPT 6
03	V5	VECTORED INTERRUPT 5
04	V4	VECTORED INTERRUPT 4
05	V3	VECTORED INTERRUPT 3
06	V2	VECTORED INTERRUPT 2
07	V1	VECTORED INTERRUPT 1
08	V0	VECTORED INTERRUPT 0
09	INT	INTERRUPT
11	DAV	
13	TBE	
15	DRDY	

VIO-X
KEYBOARD INPUT CONNECTOR (J1)

DB-25S PIN#	SIGNAL NAME	DESCRIPTION
22	DI7	DATA BIT 7
21	DI6	DATA BIT 6
20	DI5	DATA BIT 5
19	DI4	DATA BIT 4
18	DI3	DATA BIT 3
17	DI2	DATA BIT 2
16	DI1	DATA BIT 1
15	DI0	DATA BIT 0
24	RDY	DATA READY (STROBE)
23	RST	RESET
01	GND	GROUND
12	+5	+5VDC
14	+12	+12VDC (OPT)
11	SPK	SPEAKER OUT
25	-12	-12VDC (OPT)

VIO-X
CRT VIDEO CONNECTOR (J3)

PIN #	SIGNAL	DESCRIPTION
01	HOR	HORIZONTAL SYNC
02	GND	
03	GND	
04	VER	VERTICAL SYNC
05	GND	
06	VID	VIDEO OUT
07	GND	
08	GND	
09	CVID	COMPOSITE VIDEO
10	N/C	
11	N/C	
12	GND	
13	LPN	LIGHT PEN INPUT



FULCRUM COMPUTER PRODUCTS

1771 JUNCTION AVENUE • SAN JOSE, CA 95112
TELEPHONE (408) 295-7171

Distributed by WW COMPONENT SUPPLY INCORPORATED